Task Forge

AI 辅助开发使用说明文档

课程名称	软件工程
项目名称	Task Forge ·本地优先任务管理系统
技术栈	PyQt6 ·SQLAlchemy 2.0 ·SQLite
学期	2025–2026 学年第二学期

本文档记录 Task Forge 项目中 AI 工具的使用情况，
涵盖工具选型、使用环节、提示词策略、内容评价与经验总结。

目录

1 课程要求对照 **2**

2 功能展示（对应作业要求 4.1） **2**
- 2.1 功能展示证据映射（用于 4.13 现场讲解） . . . 3
- 2.2 项目文件结构（按截图中的提交要求） . . . 3

3 AI 辅助开发：工具与版本（要求 a） **3**
- 3.1 主要使用工具 . . . 3

4 AI 辅助开发：使用步骤与环节（要求 b） **4**
- 4.1 使用环节总览 . . . 4
- 4.2 核心环节详述 . . . 4
  - 4.2.1 架构设计阶段 . . . 4
  - 4.2.2 父子 DAG 联动设计 . . . 4
  - 4.2.3 回归测试套件设计 . . . 4

5 AI 辅助开发：交互方式与提示词（要求 c） **5**
- 5.1 交互方式说明 . . . 5
- 5.2 提示词示例 . . . 5
  - 5.2.1 架构设计类 . . . 5
  - 5.2.2 BUG 排查类 . . . 5
  - 5.2.3 代码生成类 . . . 6
  - 5.2.4 测试断言类 . . . 6
- 5.3 提示词使用规律 . . . 6

6 AI 生成内容评价（要求 d） **7**
- 6.1 总体评价 . . . 7
- 6.2 优点 . . . 7
- 6.3 不足与缺陷 . . . 7

7 经验总结与问题解决（要求 e） **8**
- 7.1 遇到的困难与应对方案 . . . 8
  - 7.1.1 困难一：生成代码与现有架构不兼容 . . . 8
  - 7.1.2 困难二：提醒调度路径的测试复杂性 . . . 8
  - 7.1.3 困难三：`forced_visible_ids` 机制的问题定位 . . . 8
- 7.2 可复用的 AI 协作流程 . . . 8
- 7.3 对 AI 辅助开发的整体认识 . . . 9

附录：核心提示词模板速查 **9**

1 课程要求对照

截图中的硬性条目	本文档对应位置	状态
功能展示	第 2 节（功能点 + 演示素材映射）	已覆
AI 使用说明 a）工具与版本	第 3 节	已覆
AI 使用说明 b）使用步骤/环节/模块	第 4 节	已覆
AI 使用说明 c）交互方式与提示词模板	第 5 节	已覆
AI 使用说明 d）AI 输出评价（可用性/优点/缺陷）	第 6 节	已覆
AI 使用说明 e）经验、困难与解决方式	第 7 节	已覆
项目提交结构（`src/data/README/requirements/docs`）	第 2 节中的“项目文件结构”小节	已覆

表 1: 课程作业要求与本文档内容的对应关系

2 功能展示（对应作业要求 4.1）

Task Forge 是一款基于 **PyQt6 + SQLAlchemy 2.0 + SQLite** 的本地优先（Local-First）单体桌面任务管理系统，面向个人知识工作者，核心特性如下：

- 父子 **DAG** 联动：有向无环图递归状态同步（`_refresh_parent_chain`），全部子任务完成时父节点自动归档，状态零漂移
- 精准提醒引擎：Single-shot 定时器 + `reminder_sent` 防重复互斥锁 + `postpone_reminder` 延期补偿，每条提醒只响一次
- **11** 种视图模式：今日视图 / 计划视图 / 任务树 / 甘特图 / 日历 / 看板 / 分析 Hub 等，`switch_view()` 统一路由
- 自动 **schema** 迁移：`_ensure_schema()` 启动时自动补列，历史库路径自动升级
- 深度分析能力：七边雷达图·52 × 7 GitHub 风热力图·分类分布偏差分析（$\Delta T = T_{\text{actual}} - \hat{T}$）
- 双回归测试套件：编辑器套件 + 主窗口套件，共 15+ 断言，覆盖核心执行路径

项目合计约 **8,000** 行 Python 代码，涵盖 `src/`（核心逻辑）、`src/ui/`（25 个 UI 模块）、`tests/`（测试套件）三个主要模块区域。

2.1 功能展示证据映射（用于 4.13 现场讲解）

课程功能点	实现说明	演示素材
基础界面与操作	主窗口 + 任务编辑器 + 列表视图 + 完成/删除操作	`docs/demo/01-today-view.p`
数据持久化	SQLite 本地存储，启动自动迁移与自动备份	`docs/demo/10-export-impor`
父任务与子任务	DAG 联动，子任务全完成后父任务自动归档	`docs/demo/03-parent-child`
任务提醒	Single-shot 调度 + 防重复提醒锁 + 延期补偿	`docs/demo/08-reminder-dia`
可视化扩展能力	甘特图/日历/看板/分析 Hub 多视图联动	`docs/demo/04-gantt-view.p`

表 2: 功能点与演示证据对应表

2.2 项目文件结构（按截图中的提交要求）

Listing 1: 提交目录建议（最小集合）

```
Task-Forge/
|-- src/                      # 源代码
|-- data/                     # 数据文件
|-- README.md                 # 项目说明
|-- requirements.txt          # 依赖列表
`-- docs/                     # AI 使用说明文档
  |-- AI使用说明.tex
  |-- AI使用说明.pdf
  `-- fonts/                # README/TeX 所需字体
    |-- NotoSansSC-VF.ttf
    `-- NotoSerifSC-VF.ttf
```

3 AI 辅助开发：工具与版本（要求 a）

3.1 主要使用工具

工具	接入方式	主要用途	版本/模型
GitHub Copilot Chat	VS Code 扩展	架构设计、代码生成、排查 BUG、文档撰写	GPT-5.3-Codex
GitHub Copilot 补全	VS Code 内联	函数体自动补全、变量名建议	Copilot (inline)

表 3: 项目使用的 AI 辅助开发工具一览

主要使用模型为 **GPT-5.3-Codex**（复杂实现、重构与排错），辅以 **GPT-4o**（快速问答与格式转换）。Copilot 内联补全在几乎所有文件中均有激活。

4 AI 辅助开发：使用步骤与环节（要求 b）

4.1 使用环节总览

阶段	环节	**AI** 辅助内容
1	需求分析与架构设计	五层架构划分、单向数据流规范、DAG 状态机形式化
2	数据库层（`DB.py`）	ORM 实体定义、`_ensure_schema` 迁移方案、`_refresh_parent_chai`
3	核心 UI 框架	`MainWindow` 事件路由、提醒调度时序、`refresh_everything` 统一刷新
4	任务编辑器	`set_context` 三模式注入、`StarryTagEditor` 流式布局、时间约束联动
5	多视图模块	甘特图 `paintEvent` 自绘、热力图渲染、雷达图七轴布局
6	回归测试套件	测试断言设计、异常路径识别、`reminder_sent` 防重复路径验证
7	演示数据植入	`data_seeder.py` 完整多层级数据生成方案
8	文档撰写	README 架构说明、LaTeX 报告、GIF 录制操作说明

表 4: AI 辅助开发全周期使用环节梳理

4.2 核心环节详述

4.2.1 架构设计阶段

项目初期使用 AI 对五层架构（表现层 / 交互层 / 领域层 / 数据层 / 资源层）进行了设计确认和技术选型讨论。AI 帮助形式化了单向数据流约束——即所有 UI 状态变更必须经由 `DB.py` 接口落盘后，再通过 `refresh_everything()` 广播刷新。这一约束在后续所有模块开发中严格执行，最终消除了 State Drift 竞态问题。

4.2.2 父子 **DAG** 联动设计

`_refresh_parent_chain` 是项目最复杂的业务逻辑之一。AI 协助设计了三条核心不变量（全量完成 → 自动归档 / 任意未完成 → 强制回退 / 向上递归至根节点），并指出了 `reminder_sent` 字段的隔离需求——父节点状态刷新不应重置 `reminder_sent`，否则会导致已触发的父任务提醒被再次触发。该问题在早期测试中确实复现为 BUG，AI 提供的排查思路直接定位了原因所在。

4.2.3 回归测试套件设计

AI 协助识别了多个需要覆盖的测试场景：

- `reminder_timer.isSingleShot() == True`：防止定时器被误配置为重复触发
- `forced_visible_ids` 可见性断言：验证子任务在受限过滤视图中创建后仍可见
- Windows 平台 Toast 禁用断言：确认 `_should_show_system_reminder_toast() == False`
- 甘特图进度同步：验证 `gantt_entries()` 返回值与 ORM 字段严格一致

5 AI 辅助开发：交互方式与提示词（要求 c）

5.1 交互方式说明

本项目主要通过 **GitHub Copilot Chat** 对话窗口与 AI 进行交互，辅以 Copilot 内联补全。采用持续上下文对话方式——在同一对话线程内积累项目背景，减少每次重新描述的成本。

5.2 提示词示例

5.2.1 架构设计类

Listing 2: 架构设计类提示词（提炼自实际对话）

```
你是一个 Python + PyQt6 桌面应用架构师。
项目背景：Task Forge 是一款 Local-First 单体任务管理桌面应用，
数据层使用 SQLAlchemy 2.0 + SQLite，UI 层为 PyQt6。

请设计一个「父子任务 DAG 联动」机制，要求：
1. 全部子任务完成时，父任务自动标记为完成（递归向上）
2. 任意子任务取消完成时，父任务强制回退为未完成
3. completed_at 字段的写入时机
4. reminder_sent 字段不能被父节点刷新影响（给出理由）

请给出核心函数签名和状态机伪代码。
```

Listing 3: 单向数据流约束设计提示词

```
我需要确保 Task Forge 的所有视图状态变更都经过 DB 持久化，
不出现「界面先变、落盘失败」的状态漂移问题。

请为以下场景设计一个不变量：
- 用户勾选任务完成 → UI 更新时序
- 父子联动触发 → 刷新时序
- 提醒弹窗响应 → 状态写入时序

核心约束：refresh_everything() 是唯一的视图重绘入口。
```

5.2.2 BUG 排查类

Listing 4: BUG 排查类提示词示例

```
Task Forge 出现以下问题：
在 today 视图下创建子任务（set_context 传入 preferred_parent_id），
保存成功后该子任务在 UI 树中消失，但数据库有记录。

已知信息：
- today 视图按 due_at = today 过滤
- 子任务的 due_at 是 None（未设置截止时间）
- 过滤逻辑在 _refresh_tree() 中
```

```
请定位原因，并给出：
1. 问题的具体来源（哪一行 / 哪个条件）
2. 修复方案（不破坏现有过滤逻辑）
3. 回归断言建议
```

5.2.3 代码生成类

Listing 5: 代码生成类提示词示例（甘特图自绘）

```
请为 Task Forge 的 GanttTimeline 组件实现 paintEvent。

数据来源：gantt_entries() 返回以下格式
  [{"id": 1, "title": "...", "start": date, "end": date,
    "progress": 65, "priority": "高", "completed": False,
    "depth": 0, "parent_id": None}, ...]

绘制要求：
1. 每行高度 40px，奇偶行背景交替
2. 根据 depth 字段缩进（indent = depth * 18px）
3. 进度条：蓝色底色 + progress% 填充（绿色完成 / 红色逾期）
4. 今日竖线：红色虚线
5. 所有绘制加 try/except 保护（防崩溃）

使用 QPainter，不使用 QGraphicsScene。
```

5.2.4 测试断言类

Listing 6: 回归测试断言类提示词示例

```
请为以下场景设计 pytest 风格断言（实际运行在 PyQt6 QApplication 环境）：

场景：提醒调度完整路径
- 创建一个 remind_at = now - 1 second 的任务
- 调用 check_reminders()
- 预期：声音播放 + 弹窗展示 + reminder_sent 被设置为 True

额外约束：
1. Windows 平台不触发系统 Toast
2. reminder_timer 必须是 SingleShot 模式
3. 父节点刷新不能重置 reminder_sent

请给出完整的断言代码（含 mock 和 cleanup）。
```

5.3 提示词使用规律

效果较好的提示词具有以下共同特征：

1. 提供足够的项目背景：每个问题前先描述相关约束（如「`reminder_sent` 隔离需求」），避免 AI 给出脱离情境的通用答案
2. 明确输出格式：指定「函数签名 + 伪代码 + 断言建议」比「请帮我实现」输出质量高得多

3. 分步拆解：「甘特图全功能」拆成「数据接口 → paintEvent 骨架 → 进度条渲染 → 父子缩进」四轮对话，每次均能得到可直接参考的输出
4. 主动提供错误上下文：BUG 排查时提供「已知信息 / 已排查路径 / 当前错误输出」三要素，定位成功率显著提高

6 AI 生成内容评价（要求 d）

6.1 总体评价

本项目中 AI 生成内容的整体可用性较高，大部分核心逻辑可在轻度修改后直接采用，少数涉及 PyQt6 特殊行为（如 `QTimeEdit` 24 小时解析异常、`QScrollArea` 嵌套约束）的生成代码需要较多手工调整。

内容类型	可用率	修改量	评价说明
架构设计文档	95%	极少	形式化描述质量高，可直接采用
数据库 ORM 实体	90%	少量	字段设计合理，自引用 FK 正确
业务逻辑（DB.py）	85%	中等	核心逻辑正确，特殊情形需补充
UI 组件（PyQt6）	70%	较多	对框架特性有时存在偏差
回归测试断言	80%	少量	场景识别准确，mock 有时冗余
文档写作	90%	少量	结构完整，格式规范
演示数据生成	95%	极少	层级结构合理，数据真实感强

表 5: AI 生成内容分类评价

6.2 优点

- 架构设计水准较高：AI 能够快速理解 Local-First、单向数据流、DAG 状态机等概念，给出的设计方案在正确性和规范性方面均高于平均水平。对「所有视图更新应通过 `refresh_everything()` 唯一入口」这一约束的坚持，显著减少了后续 State Drift 问题的数量。
- 特殊情形识别较准确：在提醒调度设计中，AI 主动指出了 `reminder_sent` 的隔离需求，避免了「父节点刷新重置提醒锁」这一不易被直接发现的逻辑漏洞。在甘特图实现中，AI 主动建议对 `paintEvent` 每行绘制加 `try/except` 保护，有效防止单行数据异常导致整图崩溃。
- 文档结构化能力强：README、LaTeX 说明文档等写作任务的生成质量均接近可直接提交水平，mermaid 流程图、数学公式、Shields.io 徽章等格式化内容的输出准确率极高。

6.3 不足与缺陷

- **PyQt6** 框架特异性理解不足：AI 偶尔将 PyQt5 API 混入 PyQt6 输出（如 `exec_()` 而非 `exec()`），或对 `QTimeEdit` 的 24 小时/12 小时解析行为产生误判，需要开发者具备足够的框架基础才能识别和修正。

- 生成代码防御性不足：默认输出通常缺乏足够的 `None` 检查和类型保护，容易因极端输入数据（如空任务列表、`None` 父 ID）触发 `AttributeError`，需要在 code review 阶段系统性地补充防御代码。
- 复杂 **QSS** 样式生成效果一般：AI 生成的 QSS 在简单场景下可用，但对「选择器优先级冲突」「嵌套容器样式渗透」等问题，AI 的诊断往往不够准确，需要手工调试。
- 多轮对话上下文衰减：超长对话（30+ 轮次）后，AI 有时会遗忘早期确立的约束（如「禁止 UI 层直接实例化 Session」），需要开发者在重要决策前主动重述约束背景。

7 经验总结与问题解决（要求 e）

7.1 遇到的困难与应对方案

7.1.1 困难一：生成代码与现有架构不兼容

现象：引入甘特图视图时，AI 建议使用 `QGraphicsScene` 实现，但现有项目已在所有导航视图中统一采用 `QWidget + paintEvent` 自绘方式。直接接受 AI 方案会引入第二套渲染体系，增加日后维护的复杂度。

应对方案：在提示词中明确约束「使用 QPainter，不使用 QGraphicsScene」，并在对话开始前提供「现有视图的绘制方式」作为背景，引导 AI 在已有约束内给出兼容方案。经此调整，输出与现有架构完全兼容。

7.1.2 困难二：提醒调度路径的测试复杂性

现象：`check_reminders()` 的完整执行路径涉及定时器、数据库写入、声音播放、弹窗展示四个环节，AI 生成的测试代码难以在无实际用户界面的情况下模拟完整路径。

应对方案：将问题拆解为「单元验证 + 集成路径验证」两个层次：单元层验证每个函数的返回值和副作用（`reminder_sent` 状态），集成层在 `QApplication` 环境中运行真实 MainWindow，通过事件循环触发完整路径。AI 协助设计了 `window.reminder_timer.stop()` 的初始化保护，防止启动阶段即触发提醒弹窗。

7.1.3 困难三：`forced_visible_ids` 机制的问题定位

现象：在 `today` 视图下新建子任务后，子任务在 UI 树中消失（数据库有记录），用户以为操作失败。初次排查时将问题归因于「渲染延迟」，实际原因是「过滤逻辑把 `due_at=None` 的子任务排除」。

应对方案：向 AI 提供了「已知信息 / 已排查路径」双要素，AI 在第二轮对话中准确识别原因并建议引入 `forced_visible_ids` 集合机制。该方案将「用户上一次操作涉及的任务及其上级节点」强制纳入可见集合，有效解决了过滤视图下的可见性问题。

7.2 可复用的 AI 协作流程

基于本项目经验，提炼出以下可复用的 AI 辅助开发流程：

1. 约定先行：开发任何模块前，先与 AI 确认不可违背的架构约束（如单向数据流、Session 作用域），并在提示词中持续复述
2. 拆小后再生成：不要求 AI 一次生成复杂模块的全部代码；先设计接口和签名，再

分批实现各子功能

3. 生成即 **Code Review**：将 AI 输出视为「有经验开发者的草稿」，立即审查，重点检查 None 处理、类型断言和框架特异性用法
4. 测试驱动修正：遇到生成代码行为不符预期时，先写回归断言复现问题，再让 AI 基于具体断言给出修复方案，比「描述表现让 AI 猜」效率更高
5. 文档最后生成：等代码稳定后再生成文档，避免文档与实现频繁脱节

7.3 对 AI 辅助开发的整体认识

本项目中，AI 扮演了「随时可用的资深配对编程伙伴」的角色，而不是「替代开发者决策」的工具。对于架构设计、业务逻辑形式化、特殊情形识别、文档写作等高密度脑力任务，AI 的辅助效果显著，主要体现在大幅压缩了「从想法到可验证实现」的时间。而对于复杂 **UI** 调试、框架特异性问题、跨多对话长期上下文维护等任务，AI 的辅助有限，仍需依赖开发者自身的积累。

合理的预期是：AI 能够将个人项目的开发效率提升 **40%–70%**，前提是开发者已具备足够的基础，能够识别和修正 AI 的错误输出，并能将问题拆解为 AI 可有效处理的子任务。

附录：核心提示词模板速查

Listing 7: 通用问题排查提示词模板

```
【背景】Task Forge: PyQt6 + SQLAlchemy 2.0 + SQLite，本地优先任务管理桌面应
    用。
架构约束：所有数据写入通过 DB.py 接口；refresh_everything() 是唯一重绘入口。

【问题现象】<描述用户可观测的异常行为>

【已知信息】
- <已确认的相关代码路径>
- <已排查的可能原因>
- <当前错误输出 / 堆栈>

【期望输出】
1. 原因定位（精确到文件 + 行号或逻辑块）
2. 修复方案（不破坏架构约束）
3. 回归断言建议（防止再次出现）
```

Listing 8: 功能实现提示词模板

```
【背景】Task Forge 项目（PyQt6 + SQLAlchemy 2.0）
【任务】实现 <功能名称>

【数据接口】<DB.py 相关方法签名及返回格式>
【约束】
  - 不引入新的第三方依赖
  - 遵循现有 QSS 主题体系（不硬编码颜色）
```

```
  - 对所有 None / 空列表输入加防御处理

【期望输出】
  - 函数签名
  - 核心实现（重点部分，可省略样板代码）
  - 潜在风险点提示
```